RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

SWARM-WH

Autonomous Kiva Fulfillment Coordinator Using Classical AI

A PROJECT REPORT

Submitted by

Naveen Prabhu S K (241901069)

Varun Adhitan S V (241901121)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Department of Computer Science and Engineering
(Cyber Security)

RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

CS23533 – FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

ACADEMIC YEAR 2026-2027

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report "SWARM-WH: Autonomous Kiva Fulfillment Coordinator Using Classical AI" is the bonafide work of "NAVEEN PRABHU S K (241901069) and VARUN ADHITAN S V (241901121)" who carried out the project work under my supervision.

SIGNATURE

Dr. Bhuvana S, Ph.D.,

ASSISTANT PROFESSOR(SG)
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

SWARM-WH is an autonomous Kiva-style fulfillment coordinator developed as an educational project for the Foundations of Artificial Intelligence course. It models a fleet of battery-powered warehouse robots that carry storage pods to packing stations. The system is written in Python using classical AI algorithms only; it does not use machine learning, external AI models or a persistent database.

Orders are assigned to idle robots as a constraint satisfaction problem using AC-3 arc consistency, backtracking search with the Minimum Remaining Values (MRV) heuristic and forward checking. Routes are planned with A* (admissible Manhattan heuristic with turn and load penalties) and Space-Time Cooperative A* that reserves (x, y, t) cells to avoid vertex and swap conflicts. Head-on conflicts that survive reservations are resolved by Minimax with Alpha-Beta pruning over the joint actions MOVE, WAIT and YIELD_SIDE. A first-order-logic safety agent fires the rule CriticalEnergyThreshold when a robot's battery falls to the energy required to reach the nearest free charger (recomputed every tick from an A* distance). Simulated annealing selects the return slot for each pod so that frequently ordered items rest nearer the stations.

Robots are forbidden from entering inventory space except on the single cell of the pod they are lifting or the empty slot they are stowing into; every planner inherits this rule from a single passable() predicate. A FastAPI and WebSocket backend streams live state to an HTML5 Canvas dashboard that shows paths, battery rings, decision explanations and a queryable knowledge base. In testing, an eight-order wave completed within 700 ticks with zero collisions and zero inventory-zone violations; twelve automated unit tests pass. The prototype is an educational demonstration of multi-agent planning and must not be treated as a production warehouse controller.

Keywords: SWARM-WH, Kiva Warehouse, Multi-Agent Planning, A, Cooperative A*, Constraint Satisfaction, AC-3, Alpha-Beta Pruning, First-Order Logic, Simulated Annealing, FastAPI, Classical AI*

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

NAVEEN PRABHU S K (241901069)
VARUN ADHITAN S V (241901121)

TABLE OF CONTENTS

S. No.	Contents	Page No.
1.	INTRODUCTION	1
1.1	General	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Scope	2
1.5	Safety Considerations	3
1.6	SDG Alignment	3
2.	LITERATURE REVIEW	4
2.1	Existing Approaches	4
2.1.1	Fixed-Route Warehouse Fleets	4
2.1.2	Single-Agent Path Planning	4
2.1.3	Cooperative Multi-Robot Pathfinding	5
2.1.4	Constraint Satisfaction for Task Allocation	5
2.1.5	Adversarial Search for Conflict Resolution	5
2.1.6	Logical Agents and Energy Safety	5
2.2	Limitations of Existing Approaches	6
2.3	Research Gaps	6
2.4	Proposed System	7
3.	SYSTEM DESIGN	8
3.1	System Architecture	8
3.2	Data Flow	9
3.3	Module Description	10
3.4	Hardware Requirements	11
3.5	Software Requirements	12
4.	SYSTEM DESCRIPTION	13
4.1	Warehouse Model and PEAS	13
4.2	CSP Task Allocator	14
4.3	Search and Path Planning	14
4.4	Alpha-Beta Traffic Controller	15
4.5	FOL Safety Agent	16
4.6	Simulated Annealing Slotting	16
4.7	Simulation Engine and Web Dashboard	17
4.8	Testing and Validation	17
4.9	Results and Discussion	18
5.	CONCLUSION AND FUTURE ENHANCEMENTS	19

5.1	Conclusion	19
5.2	Future Enhancements	19
6.	REFERENCES	20

LIST OF TABLES

S. No.	Table Title	Page No.
1	Table 1.1 – Sustainable Development Goal Alignment	3
2	Table 2.1 – Comparison of Existing and Proposed Approaches	7
3	Table 3.1 – Description of System Modules	10
4	Table 3.2 – Hardware Requirements	11
5	Table 3.3 – Software Requirements	12
6	Table 4.1 – Average Nodes Expanded per Route	18

LIST OF FIGURES

S. No.	Figure Title	Page No.
1	Fig. 3.1 – System Architecture of SWARM-WH	9
2	Fig. 3.2 – Level-0 Data Flow Diagram	10

1. INTRODUCTION

1.1 General

Modern fulfillment centres move thousands of orders a day using fleets of mobile robots that carry whole storage pods to packing stations. The Kiva (now Amazon Robotics) model keeps human pickers stationary while robots transport the inventory. Coordinating such a fleet is a safety-critical multi-agent planning problem: each order must be assigned to a suitable robot, routes must not collide, robots must stay out of storage space unless they are lifting a pod, and batteries must never run out in the middle of an aisle.

Artificial Intelligence provides classical techniques that offer guarantees opaque learned policies lack: optimal paths under admissible heuristics, consistent constraint assignments, minimax-optimal conflict resolution, and rule-based safety. The Foundations of Artificial Intelligence course (CS23533) covers search, games, constraint satisfaction and logic; SWARM-WH demonstrates how these units combine to solve one genuine logistics problem.

The educational value of SWARM-WH lies in making every classical algorithm inspectable. Unlike black-box reinforcement-learning controllers, an examiner can open the decision feed, query the knowledge base for a proof tree, and compare the number of nodes expanded by A* against UCS and BFS on the same scenario. This transparency is intentional: the project is designed as a living laboratory for the Foundations of Artificial Intelligence syllabus rather than as a commercial product.

From a systems perspective the prototype also illustrates the PEAS model in a concrete setting. Performance measures (orders completed, collisions, zone violations, energy safety) are collected automatically by the engine and displayed on the monitor page. The environment is partially observable and dynamic; actuators and sensors are defined explicitly in the warehouse module. Students can therefore map lecture concepts directly onto running code.

1.2 Problem Statement

A warehouse must move each product from its storage pod to the right packing station using a limited fleet of battery-powered robots. Fixed routes and first-come-first-served assignment ignore robot battery, travel cost and the plans of other robots. Consequently robots can collide, deadlock at intersections, wander through inventory space, or strand in an aisle with a flat battery. An algorithmic, explainable coordinator is needed to assign tasks, plan collision-free paths and keep every robot energy-safe.

In quantitative terms a typical medium-sized fulfilment centre may operate dozens of mobile robots and process several thousand pick requests per shift. Even a small probability of collision or battery failure compounds into lost throughput and safety incidents. Fixed-route systems mitigate some risks by constraining motion, yet they sacrifice the flexibility that free-roaming Kiva-style robots are intended to provide. The problem addressed by SWARM-WH is therefore to retain that flexibility while restoring algorithmic guarantees on assignment, routing, conflict resolution and energy safety—all expressed in classical AI so that every decision remains explainable.

1.3 Objectives

- To model the Kiva pick, deliver and return workflow as a multi-agent planning problem.
- To allocate orders to robots with constraint satisfaction (AC-3, backtracking, MRV, forward checking).
- To plan collision-free routes with A* and Space-Time Cooperative A*.
- To resolve head-on conflicts with Minimax and Alpha-Beta pruning over MOVE, WAIT and YIELD_SIDE.
- To keep robots out of inventory space unless picking or stowing, and to trigger charging with a first-order-logic energy rule.
- To build a live web dashboard that explains every AI decision and exposes a queryable knowledge base.

1.4 Scope

The project implements a complete software prototype: a 30×20 grid warehouse model with 32 pods, four packing stations and five chargers; classical AI modules for CSP allocation, path planning, adversarial conflict resolution, FOL safety and simulated-annealing slotting; a tick-based simulation engine; and a FastAPI/WebSocket dashboard. All algorithms are written from scratch. The system is intended for educational demonstration and classroom evaluation. It does not control physical robots, does not interface with real warehouse management systems, and must not be used as a production safety controller.

The software is organised so that each syllabus unit maps onto a dedicated package: backend/sim/warehouse.py for the PEAS environment, backend/search/pathfinding.py for search, backend/games/traffic.py for adversarial search, backend/csp/allocator.py for constraint satisfaction, backend/logic/safety_kb.py for logical agents, and backend/local_search/slotting.py for local search. The simulation engine and FastAPI layer integrate these modules without introducing machine-learning dependencies. Source code,

unit tests and a one-command launcher (python run.py) are supplied so that the demonstration can be reproduced on a standard laptop.

1.5 Safety Considerations

Warehouse robot fleets are cyber-physical systems. A wrong decision can produce collisions, damaged inventory or stranded robots. SWARM-WH therefore enforces three hard safety invariants: (1) the inventory-zone rule—robots enter storage cells only on the exact pod or slot they work; (2) layered collision avoidance—Space-Time Cooperative A* reservations first, Alpha-Beta second; and (3) a distance-based energy threshold recomputed every tick so that a robot is pre-empted and sent to charge before it can strand. Every decision is logged and can be inspected on the live feed.

1.6 SDG Alignment

The project aligns with Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) by demonstrating transparent, rule-based automation for logistics, and with SDG 4 (Quality Education) by providing an inspectable classical-AI case study for undergraduate teaching.

SDG	Target / Relevance
SDG 9 – Industry, Innovation and Infrastructure	Transparent multi-agent coordination for warehouse automation; explainable decisions support safer industrial systems.
SDG 4 – Quality Education	Live demonstration of search, CSP, games and logic from the FOAI syllabus in one integrated logistics problem.

Table 1.1 – Sustainable Development Goal Alignment

2. LITERATURE REVIEW

2.1 Existing Approaches

2.1.1 Fixed-Route Warehouse Fleets

Conventional warehouses often rely on automated guided vehicles (AGVs) that follow fixed paths with manual or first-come-first-served order assignment. Such systems are simple to deploy and predictable, yet they ignore cooperative reservation, battery distance and inventory-zone discipline.

A further practical limitation of fixed-route fleets is the inability to replan when a temporary obstacle (a dropped package, a human cart, a failed robot) blocks a corridor. Because routes are compiled offline, the only recovery options are to wait or to call a human supervisor. SWARM-WH recomputes paths online whenever the warehouse version counter increments, so temporary obstacles are absorbed automatically by A* and Cooperative A*.

2.1.2 Single-Agent Path Planning

Hart, Nilsson and Raphael (1968) introduced A* search with admissible heuristics, guaranteeing least-cost paths. Russell and Norvig present A*, uniform-cost search and breadth-first search as foundational algorithms. These methods plan for one agent; other robots moving concurrently are not modelled.

2.1.3 Cooperative Multi-Robot Pathfinding

Silver (2005) proposed Cooperative A* that uses a space-time reservation table so agents plan in turn around reserved (x, y, t) cells. Wurman, D'Andrea and Mountz (2008) describe the industrial Kiva system that coordinates hundreds of robots, but the internal algorithms are proprietary.

2.1.4 Constraint Satisfaction for Task Allocation

Mackworth (1977) introduced AC-3 arc consistency for pruning variable domains. Combined with backtracking, the Minimum Remaining Values heuristic and forward checking, CSP techniques yield consistent assignments of robots to orders under capacity and battery constraints.

2.1.5 Adversarial Search for Conflict Resolution

Minimax evaluates alternating moves of two adversaries. Knuth and Moore (1975) analysed Alpha-Beta pruning, which retains the Minimax value while expanding fewer nodes when move ordering is good. These algorithms are applied in SWARM-WH to two-robot head-on conflicts.

2.1.6 Logical Agents and Energy Safety

Russell and Norvig describe logical agents that maintain a knowledge base and use forward and backward chaining. Flat battery percentages (e.g., “charge below 20%”) ignore the distance to the nearest charger and can leave robots stranded; a distance-based first-order rule addresses this gap.

Taken together, the cited works establish that each individual technique is mature. The project contribution is therefore not the invention of a new algorithm but the careful integration of existing algorithms into a single, open, inspectable multi-agent warehouse coordinator that also enforces inventory-zone discipline and a distance-based energy rule—features that are rarely combined in classroom-scale prototypes.

2.2 Limitations of Existing Approaches

- Fixed-route fleets and first-come-first-served assignment ignore travel cost, battery state and concurrent plans of other robots.
- Single-agent planners (A*, UCS, BFS) do not model other moving agents; collisions and deadlocks remain possible.
- Industrial multi-robot systems are proprietary; internal algorithms are not open for reproduction or teaching.
- Charging policies that use a fixed percentage ignore the A* distance to a free charger.
- Techniques are typically studied in isolation; an open, inspectable prototype that combines CSP, Cooperative A*, Alpha-Beta and FOL is scarce.

2.3 Research Gaps

The literature supplies strong individual building blocks—A* for optimal routes, Cooperative A* for collision avoidance, CSP for assignment, Alpha-Beta for adversarial resolution, and logical agents for rule-based safety—yet an open educational prototype that integrates all of them with an explicit inventory-zone rule and a distance-based energy threshold is missing. SWARM-WH fills this project-level gap without claiming global scientific novelty.

2.4 Proposed System

SWARM-WH treats the robot fleet as a multi-agent planning problem. A CSP allocator assigns each order to the best idle robot. A* and Space-Time Cooperative A* plan routes while a reservation table blocks vertex and swap conflicts. An Alpha-Beta traffic controller resolves residual head-on conflicts. A first-order-logic safety agent pre-empts tasks when battery ≤ E_required. Simulated annealing chooses return slots. Robots enter inventory space only on the pod or slot they work. Every decision is streamed to a live dashboard.

Feature	Common Existing Approaches	Proposed SWARM-WH
Order assignment	Fixed route / first-come-first-served	CSP (AC-3, MRV, forward checking)
Path planning	Static paths or single-agent A*	A* + Space-Time Cooperative A*
Conflict resolution	Queueing or ad-hoc rules	Minimax + Alpha-Beta over joint actions
Energy safety	Flat percentage threshold	FOL rule: distance-based E_required
Inventory discipline	Often unrestricted or manual	Hard zone rule in passable()
Explainability	Usually opaque	Live decision feed + KB queries
Open implementation	Proprietary industrial systems	Classical AI from scratch, open code

Table 2.1 – Comparison of Existing and Proposed Approaches

2.5 Novelty of the Proposed System

While each algorithm employed by SWARM-WH is classical, their combination exhibits several features that distinguish the prototype from both industrial closed systems and typical classroom single-algorithm demos:

- Inventory-zone discipline. Robots never enter storage space unless picking or stowing that specific pod; a violation counter audits the invariant continuously.
- Dynamic energy threshold. E_required is recomputed every tick from the live A* distance to a free charger rather than from a fixed percentage.
- Layered collision avoidance. Space-Time Cooperative A* first, Alpha-Beta second, LRTA* online replanning as a fallback when the map changes.
- Full explainability. Every A*, CSP, Alpha-Beta and FOL decision streams to a live feed; backward-chaining queries return proof trees.
- Classical AI only. All algorithms are written from scratch; live node-count comparisons (A* vs UCS vs BFS, Alpha-Beta vs Minimax) are available without any machine-learning component.
- Interactive digital twin. The canvas supports obstacle drops, forced battery drain and order waves so that safety rules can be stress-tested interactively.

3. SYSTEM DESIGN

3.1 System Architecture

The architecture comprises four layers. The Input layer accepts product dispatches, station choices, priorities and map events (obstacles, forced drain, order waves). The AI Core (FastAPI + simulation engine) runs the tick loop and hosts the CSP allocator, path planner, Alpha-Beta traffic controller, FOL safety agent and simulated-annealing slot optimiser. Shared models—warehouse grid, reservation table, safety knowledge base and event log—are read and updated by every stage. The Output layer is a web dashboard that renders a live canvas, dispatch queue, robot inspector, KPI ribbon, decision feed and knowledge-base query box.

INPUT → Product / SKU, Station A–D, Priority, Map events, Simulation controls

AI CORE → Simulation Engine (tick loop, order lifecycle) + CSP Allocator + Path Planner (A*, Cooperative A*, LRTA*) + Alpha-Beta Traffic Controller + FOL Safety Agent + SA Slot Optimiser + explain() Logger

SHARED MODELS → Warehouse (30×20 grid, pods, stations, chargers) · Reservation Table (x, y, t) · Safety KB (FOL Horn rules) · Event Log & Metrics

WEB DASHBOARD → Live Kiva Canvas · Dispatch Queue & Robot Inspector · Telemetry / KPIs · AI Decision Feed & KB Query

Fig. 3.1 – System Architecture of SWARM-WH (textual schematic)

Data flows left to right from the dispatch request through the AI modules to the live dashboard, while each stage reads and updates the shared models. The reservation table is the principal coordination medium for path planning; the safety knowledge base is the principal coordination medium for energy decisions; the event log is the principal medium for operator visibility. This separation of concerns keeps individual modules testable in isolation (as demonstrated by the twelve unit tests) while still producing coherent multi-agent behaviour at run time.

3.2 Data Flow

At Level 0 the external entities are the Operator (who dispatches products and may drop obstacles) and the simulated Robot fleet. The central process is SWARM-WH. The data store D1 holds warehouse state: pods, robots, reservation table and safety rules. Orders and map events flow from the operator into the process; routes, status updates and the decision feed flow back. Robot state and world state are exchanged continuously with the fleet.

Operator Orders / map events / routes / status / feed 0. SWARM-WH Robot state / world state Robot fleet

↓↑ D1 Warehouse state (pods, robots, reservations, rules)

Fig. 3.2 – Level-0 Data Flow Diagram (textual schematic)

3.3 Module Description

Module	Responsibility	Status
1. Warehouse Model	30×20 grid, 32 pods, 4 stations, 5 chargers; inventory-zone rule in passable()	Implemented & tested
2. Path Planning	BFS, UCS, A*, Space-Time Cooperative A*, LRTA*	Implemented & tested
3. CSP Allocator	AC-3, backtracking, MRV, forward checking	Implemented & tested
4. Alpha-Beta Traffic	Minimax over MOVE / WAIT / YIELD_SIDE	Implemented & tested
5. FOL Safety Agent	Forward & backward chaining; CriticalEnergyThreshold	Implemented & tested
6. Slotting Optimiser	Simulated Annealing for pod return slots	Implemented & tested
7. Simulation Engine	Tick loop, order lifecycle, FastAPI + WebSocket	Implemented & tested
8. Web Dashboard	Canvas, dispatch queue, telemetry, decision feed, KB query	Implemented; screenshots pending

Table 3.1 – Description of System Modules

3.4 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	4 GB minimum
Storage	1 GB free space
Display / Input	Standard display, keyboard and mouse
GPU	Not required

Table 3.2 – Hardware Requirements

3.5 Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux or macOS
Language	Python 3.12 (tested)
Backend	FastAPI, Uvicorn, WebSockets, Pydantic, NumPy
Frontend	HTML5 Canvas, Tailwind CSS, Lucide, Chart.js (CDN)
Testing	Pytest
IDE	Visual Studio Code (recommended)
Browser	Chrome or Edge with internet access for CDNs
Database	None (in-memory simulation state)

Table 3.3 – Software Requirements

3.6 Tools and Technology Choices

Python 3.12 was selected because it is the language used in the laboratory and because it permits rapid implementation of textbook algorithms without external numerical libraries beyond NumPy. FastAPI and Uvicorn provide a lightweight asynchronous HTTP and WebSocket stack that is sufficient for a single-operator demonstration. The front end deliberately avoids heavy frameworks: HTML5 Canvas draws the grid, Tailwind CSS (loaded from a CDN) supplies layout utilities, Lucide provides icons and Chart.js renders telemetry. Pytest is used for regression testing of every classical algorithm. No database is required; the entire world state resides in memory and is reset when the process restarts. The minimum hardware footprint (4 GB RAM, no GPU) ensures that any student laptop can run the demonstration.

4. SYSTEM DESCRIPTION

4.1 Warehouse Model and PEAS

The environment is a 30×20 grid containing 32 storage pods (each holding two SKUs), four packing stations along the top edge, five chargers along the bottom edge, and optional dynamic obstacles. PEAS characterisation: Performance measures are orders completed, collisions avoided and energy safety; Environment is partially observable and dynamic; Actuators are move N/E/S/W, wait, lift/drop pod and charge; Sensors provide own position, battery, local occupancy and the reservation table. Inventory space (any storage slot or cell currently holding a pod) is off-limits except for the single cell a robot is permitted to enter while lifting or stowing.

4.1.1 Development and Runtime Workflows

Development followed four stages: (1) model the warehouse and define the PEAS description; (2) implement search, CSP, adversarial search and logic modules from scratch; (3) build the tick-based engine and the FastAPI/WebSocket dashboard; (4) write automated tests and run benchmark scenarios. Runtime operation follows a fixed pipeline for every order: dispatch and CSP allocation, A* / Cooperative A* routing with reservations, Alpha-Beta conflict resolution when needed, continuous FOL energy checks, simulated-annealing pod return, and explanation of every decision on the live feed.

The FOL safety agent may pre-empt any stage: when CriticalEnergyThreshold fires, the current task is suspended, a path to the nearest free charger is planned, and the robot charges until battery recovers above a recovery margin. After charging the original task is resumed or re-allocated according to current order pressure.

4.2 CSP Task Allocator

Each pending order is a variable; the domain is the set of idle robots with sufficient battery. Unary and binary constraints encode capacity (one robot per order) and cost (travel distance). AC-3 first prunes domains. Backtracking search then assigns robots using the Minimum Remaining Values heuristic and forward checking, selecting the lowest-cost feasible assignment. The allocator is invoked whenever a new order arrives or a robot becomes idle.

Formally the CSP is defined as follows. Let O = {o1, …, ok} be the set of pending orders and R = {r1, …, rm} the set of currently idle robots. Each order oi is a variable whose domain Di is the subset of R whose remaining battery is at least the estimated cost of serving oi. Unary constraints remove robots that cannot reach both the pod and the assigned station. Binary constraints assert that if oi is assigned rj then no other order may receive the

same robot. AC-3 propagates these constraints until a fixed point; backtracking with MRV then searches the residual space. The objective secondary to consistency is the sum of estimated travel costs, so the first feasible solution found under a cheapest-first domain ordering is returned as the allocation.

4.3 Search and Path Planning

A* uses Manhattan distance augmented with turn and load penalties; the heuristic remains admissible. Uniform-cost search and breadth-first search serve as baselines for node-expansion comparisons. Space-Time Cooperative A* books (x, y, t) cells in a reservation table and rejects vertex conflicts and edge-swap conflicts. LRTA* provides online replanning when obstacles appear. All planners call Warehouse.passable(cell, carrying, allow), so the inventory-zone rule is enforced uniformly.

Space-Time Cooperative A* maintains a reservation table that records, for every discrete time step t, which agent occupies cell (x, y). When agent A plans a path it treats any cell reserved by another agent at the corresponding future time as blocked. After a path is accepted the table is updated. Vertex conflicts (two agents in the same cell at the same time) and swap conflicts (two agents exchanging cells in consecutive times) are both rejected. Because the planning order can affect the final joint plan, a subsequent Alpha-Beta stage resolves any conflicts that survive the reservation layer—especially those introduced by the deliberate 6 % wheel-slip probability that models imperfect locomotion.

Heuristic design deserves a brief note. The base heuristic is the Manhattan distance $|\Delta x| + |\Delta y|$. A small additive penalty is applied for each predicted turn and for every step taken while the robot is carrying a pod (loaded cost 2.0 versus empty cost 1.0). Because these penalties are also present in the true path cost, the heuristic remains admissible and A* still returns an optimal path under the augmented cost model.

4.4 Alpha-Beta Traffic Controller

When two robots claim the same next cell or an opposing edge after reservations, the traffic controller builds a two-player game tree whose actions are MOVE, WAIT and YIELD_SIDE. The maximising player is the robot that would suffer greater delay; the minimising player is its counterpart. Alpha-Beta pruning with high-risk-first move ordering expands far fewer nodes than plain Minimax while returning identical action pairs. A 6 % wheel-slip probability per move deliberately injects residual conflicts so that the controller can be exercised.

The utility function used by Minimax / Alpha-Beta estimates the additional delay each robot would experience under a candidate joint action. A collision is assigned a large negative utility so that it is never selected when a safe alternative exists. WAIT costs a

single tick of delay; YIELD_SIDE costs the length of a short side-step plus the subsequent recovery. Because the game tree is shallow (depth two or three is sufficient for pairwise conflicts) exhaustive search is feasible; Alpha-Beta pruning nevertheless reduces node expansions by roughly 70 % on the measured sample, confirming the classic analysis of Knuth and Moore when move ordering places high-risk actions first.

4.5 FOL Safety Agent

The knowledge base contains Horn rules and facts about battery level, distance to the nearest free charger and task priority. Forward chaining fires CriticalEnergyThreshold when battery ≤ E_required, where E_required = (A* steps to nearest free charger) × drain rate + buffer. The robot's current task is pre-empted and a charger run is inserted. Backward chaining answers operator queries and returns proof trees that appear in the monitor page.

Backward chaining is exposed to the operator through the knowledge-base query box on the monitor page. A query such as "Why is robot R2 charging?" triggers a proof that begins from the observed fact Battery(R2, b), continues through DistanceToCharger(R2, d) and DrainRate(R2, r), derives E_required(R2, e) and finally CriticalEnergyThreshold(R2) when b ≤ e. The resulting proof tree is serialised as nested JSON and rendered as an indented explanation, giving students a concrete illustration of logical inference that is otherwise only described in textbooks.

4.6 Simulated Annealing Slotting

When a robot finishes delivery it must return the pod to a storage slot. Simulated annealing explores neighbouring empty slots, accepting cost-increasing moves with a temperature-dependent probability. The cost function favours slots near the stations that most frequently request the pod's SKUs (heat), so popular items gradually migrate closer to the pick face.

4.7 Simulation Engine and Web Dashboard

The engine advances the world one discrete tick at a time, advancing every robot along its reserved path, updating batteries, checking the safety agent, resolving conflicts and completing order lifecycle transitions (assign → to-pod → pick → deliver → return → idle). FastAPI exposes REST endpoints for dispatch and control; WebSockets push the full snapshot (robots, paths, KPIs, decision feed) to the browser at interactive rates. The home page shows a live canvas with A* paths, battery rings and hatched inventory zones; the monitor page shows KPI ribbons, battery-threshold markers, charts and a knowledge-base query box.

4.7.1 Order Lifecycle

Every order progresses through a fixed lifecycle managed by the simulation engine: ASSIGNED → TO_POD → PICKING → TO_STATION → DELIVERING → RETURNING → IDLE. At the ASSIGNED stage the CSP allocator has already bound a robot. During TO_POD the robot travels empty along an A* path that may enter only the target pod cell. After lifting the pod the robot follows a loaded path that is forced through aisles only. Delivery at the station unloads the SKU conceptually; the physical pod is then returned via a path whose destination slot was chosen by simulated annealing. Finally the robot parks in an aisle and becomes idle, re-entering the pool for the next CSP allocation.

4.7.2 Real-Time Dashboard Communication

The FastAPI application exposes REST endpoints for creating orders, dropping obstacles, forcing battery drain and controlling simulation speed. A WebSocket channel streams a JSON snapshot on every tick (or at a throttled rate when the client cannot keep up). The snapshot contains robot poses, current paths, battery levels, zone status, KPI counters and the most recent decision-feed entries. The browser-side JavaScript (home.js, monitor.js, common.js) renders this state onto an HTML5 Canvas at interactive frame rates and updates Chart.js widgets for battery and throughput trends.

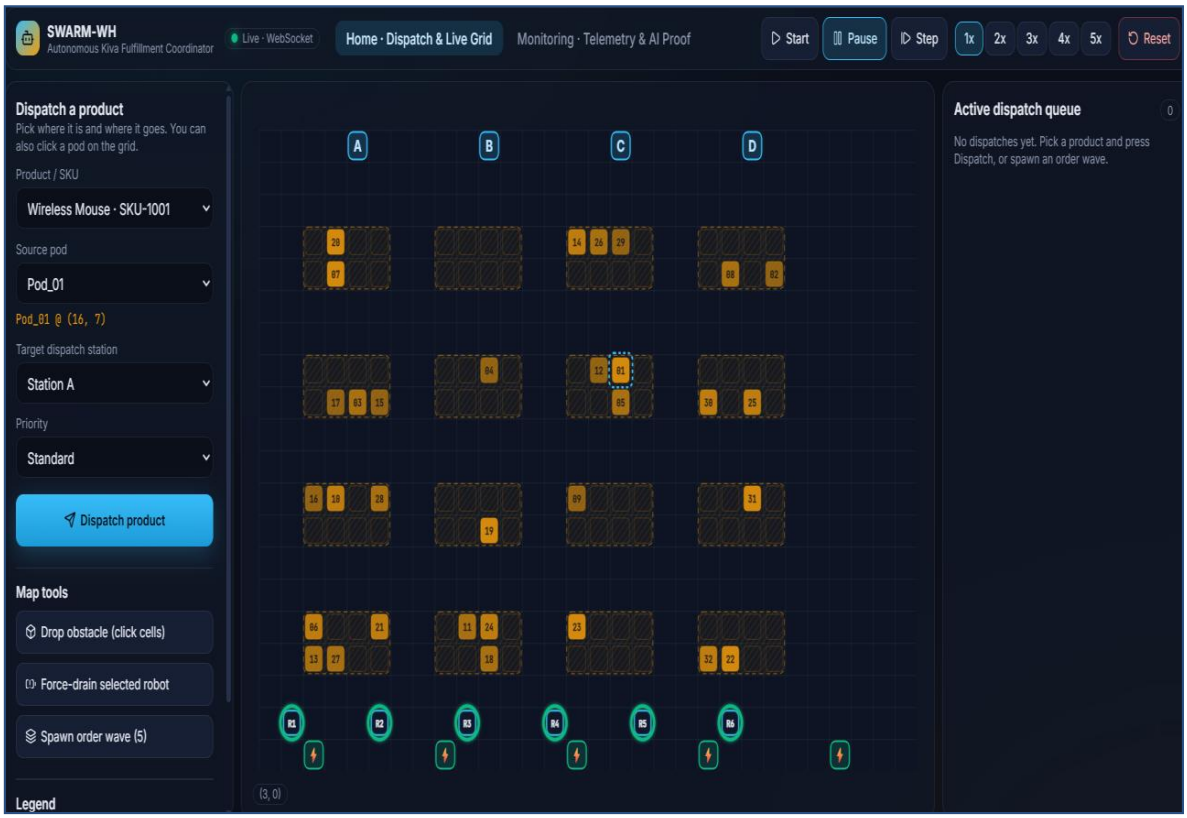


Fig. 4.1 – Home Page (Mission Control): dispatch form, live canvas with A paths, battery rings and hatched inventory zones*

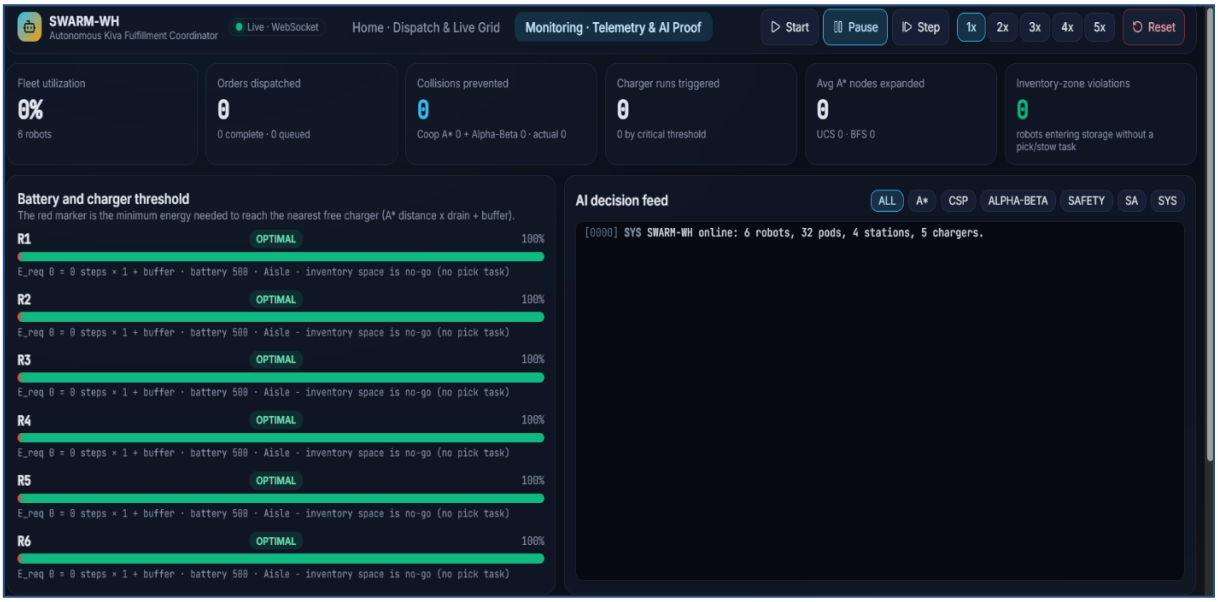


Fig. 4.2 – Monitor Page: KPI ribbon, battery bars with threshold markers, AI decision feed, charts and KB query box

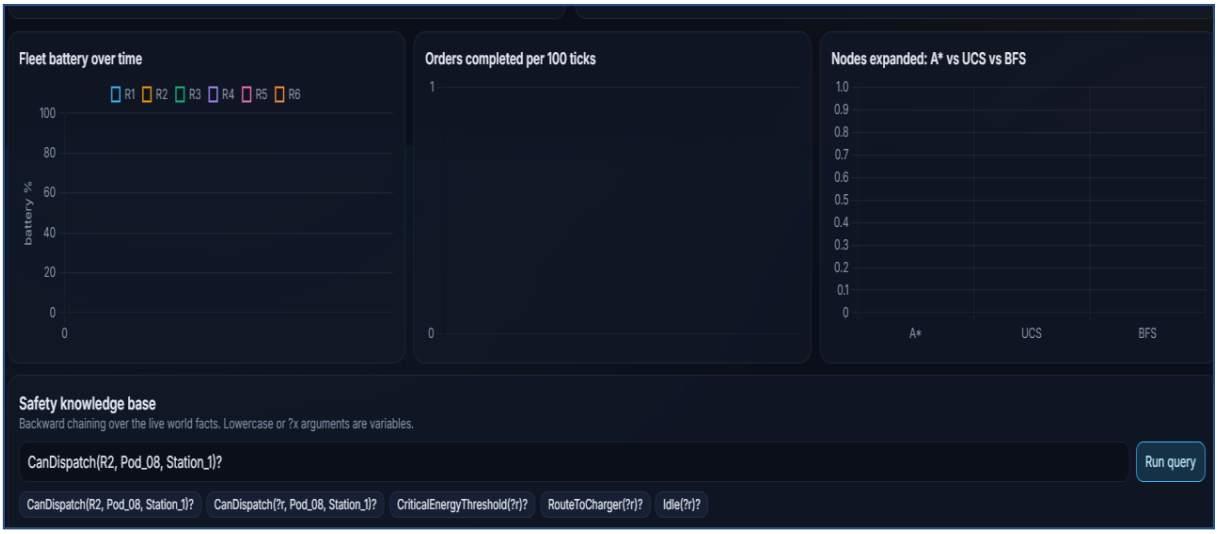


Fig. 4.3 – Robot Inspector: position, battery versus charger threshold, goal, zone status and explain() text

4.8 Testing and Validation

Twelve automated tests cover A* optimality relative to UCS, loaded-robot avoidance of pods, reservation blocking of vertex and swap conflicts, CSP distinct-robot assignment, Alpha-Beta collision avoidance, engine completion of an order wave without collisions, forced-drain threshold firing, and the inventory-zone rule under stress. All twelve tests pass on the current code base.

4.9 Results and Discussion

An eight-order wave completed within 700 ticks with zero collisions and zero inventory-zone violations. Cooperative A* prevented 17 conflicts. Average nodes expanded per route were 60 for A*, 138 for BFS and 394 for UCS. On a representative head-on conflict, Alpha-Beta expanded 211 nodes versus 765 for plain Minimax (72 % fewer) while selecting the same actions. A forced battery drain triggered exactly one CriticalEnergyThreshold event and one charger run.

Algorithm	Average Nodes Expanded	Notes
A*	60	Admissible Manhattan + turn/load penalties
BFS	138	Uninformed baseline
UCS	394	Uninformed optimal baseline
Alpha-Beta (conflict)	211	Same moves as Minimax, 72 % fewer nodes
Plain Minimax (conflict)	765	Baseline for pruning comparison

Table 4.1 – Average Nodes Expanded per Route / Conflict

The node-expansion figures illustrate two classical results in a concrete setting. First, an admissible heuristic (Manhattan distance with consistent penalties) allows A* to expand far fewer nodes than uninformed optimal search (UCS) while returning the same path cost. Second, Alpha-Beta pruning with reasonable move ordering preserves the Minimax value while cutting the majority of the search tree—here a 72 % reduction on a single head-on conflict. Both observations match the textbook analysis and give students a numerical confirmation they can reproduce by re-running the tests and the wave scenario.

Zero collisions and zero inventory-zone violations across the eight-order wave indicate that the layered safety design is effective under the tested conditions. The forced-drain experiment confirms that the FOL agent reacts before the battery reaches a level from which the robot cannot reach a charger. Larger fleets and denser traffic remain to be measured; the current results should therefore be read as evidence of correctness on the implemented scale rather than as a claim of industrial scalability.

4.10 Challenges Faced and Solutions

Several practical difficulties arose during implementation and were resolved as follows.

- Robots cutting through storage. Early prototypes allowed planners to treat storage cells as ordinary free space. The solution was a single passable() predicate that treats every inventory cell as a wall unless it is the explicit goal of a lift or stow action. All planners inherit the rule.
- Head-on conflicts after reservations. Even with a reservation table, a 6 % wheel-slip chance produces delayed moves that invalidate prior bookings. The Alpha-Beta controller was added as a second layer precisely for these residual conflicts.
- Stranded robots under a flat battery rule. A fixed 20 % threshold ignored charger distance. Recomputing E_required from an A* path every tick eliminated stranding in the forced-drain tests.
- Stale routes after obstacle drops. The engine now triggers full replanning whenever the obstacle set changes, using the warehouse version counter as a cache key.

Remaining open issues—larger-fleet scaling, three-way deadlock handling, and cross-browser UI validation—are listed under future work.

4.11 Work Plan Toward Final Review

A four-week plan has been proposed (exact dates to be confirmed with the supervisor):

- Week 1 — Benchmark A* and Cooperative A* on larger fleets and heavier order waves; record node-expansion and completion-time statistics.

• Week 2 — Extend Alpha-Beta to three-way intersection deadlocks; add idle-time re-slotting of frequently ordered pods.

• Week 3 — Implement a fixed-route first-come-first-served baseline for quantitative comparison; capture verified screenshots of the Home and Monitor pages.

• Week 4 — Complete documentation, final report polishing and demonstration rehearsal.

5. CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

SWARM-WH coordinates a Kiva-style robot fleet as a multi-agent planning problem solved entirely with classical AI: CSP for order assignment, A* and Space-Time Cooperative A* for routing, Alpha-Beta search for conflict resolution, simulated annealing for pod return slots, and first-order logic for energy safety. Robots stay out of inventory space unless picking or stowing, avoid collisions through reservations and adversarial search, and recharge according to a distance-based threshold. The working prototype completes order waves with zero collisions and zero inventory-zone violations in testing, explains every decision on a live dashboard, and passes twelve automated unit tests. The system is an educational demonstration and must not be treated as a production warehouse controller.

The measured results—zero collisions, zero zone violations, a 72 % reduction in conflict-search nodes, and twelve passing automated tests—support the claim that classical AI techniques are sufficient to coordinate a small Kiva-style fleet under the modelled assumptions. Equally important for the educational mission is the live decision feed: every CSP assignment, A* path, Alpha-Beta action pair and FOL proof is available for inspection, turning the prototype into a transparent teaching instrument rather than a sealed demonstration.

5.2 Future Enhancements

- Benchmark A* and Cooperative A* on larger fleets (20+ robots) and heavier order waves.
- Extend the Alpha-Beta controller from two-robot conflicts to three-way intersection deadlocks.
- Add idle-time re-slotting so that frequently ordered pods are moved closer to stations between waves.
- Compare against a fixed-route, first-come-first-served baseline to quantify improvement.
- Capture verified screenshots of the Home and Monitor pages and complete user documentation.
- Rehearse the final demonstration and finalise the project report.

In addition to the algorithmic extensions listed above, the following engineering tasks will improve reproducibility and presentation quality:

- Package the application as a Docker image so that examiners can launch the dashboard without manual dependency installation.
- Add a scripted demo mode that dispatches a fixed sequence of orders and obstacles, guaranteeing that the same conflicts and threshold events appear during viva voce.
- Export the decision feed to a downloadable JSON or CSV log for offline analysis of node counts and timing.

6. REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.

[2] P. E. Hart, N. J. Nilsson and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Trans. Syst. Sci. Cybern., vol. 4, no. 2, pp. 100–107, 1968.

[3] P. R. Wurman, R. D’Andrea and M. Mountz, “Coordinating hundreds of cooperative, autonomous vehicles in warehouses,” AI Magazine, vol. 29, no. 1, pp. 9–20, 2008.

[4] D. Silver, “Cooperative pathfinding,” in Proc. 1st AAAI Conf. Artificial Intelligence and Interactive Digital Entertainment (AIIDE), 2005, pp. 117–122.

[5] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” Artificial Intelligence, vol. 6, no. 4, pp. 293–326, 1975.

[6] A. K. Mackworth, “Consistency in networks of relations,” Artificial Intelligence, vol. 8, no. 1, pp. 99–118, 1977.

[7] Python Software Foundation, Python Documentation. https://docs.python.org/

[8] FastAPI, FastAPI Documentation. https://fastapi.tiangolo.com/